Guia de estudo de Batalha Naval em Python

Explicação do código e glossário dos recursos da linguagem

Material de estudo para Pedro Alvimm. O objetivo é compreender o que cada parte faz, por que foi escrita e como explicar as decisões durante uma apresentação. A numeração acompanha a versão dos três arquivos analisada nesta conversa.

Como estudar

Comece pelos conceitos, acompanhe o código comentado na ordem proposta e consulte o glossário quando surgir uma palavra desconhecida. Depois, explique em voz alta o caminho de um tiro e responda às perguntas de revisão sem olhar as respostas.

Conteúdo: 1 Conceitos e arquitetura; 2 Arquivo principal; 3 Regras do jogo; 4 Interface; 5 Glossário completo; 6 Fluxo de um ataque; 7 Apresentação e revisão.

1 Conceitos e arquitetura

Conceito	Significado no projeto
Classe	Modelo que reúne dados e comportamentos. Navio, Tabuleiro, Jogador e Computador são classes.
Objeto ou instância	Um elemento concreto criado a partir de uma classe, como um determinado cruzador.
Atributo	Dado guardado em um objeto, como self.nome ou self.tamanho.
Método	Função definida dentro de uma classe, como atacar ou receber_tiro.
Função	Bloco reutilizável de código. Neste guia, o nome também distingue as funções fora das classes.
self	Referência ao próprio objeto que executa um método. É uma convenção de nome, não palavra reservada.
Herança	Reutilização do comportamento de uma classe base. Computador herda de Jogador.
Módulo	Arquivo Python importável. O projeto separa inicialização, regras e interface.

batalha_naval.py inicia o menu e reexporta nomes. jogo.py concentra objetos e regras sem pedir input nem mostrar menus. interface.py apresenta o jogo, recebe escolhas, coordena a partida e salva o histórico. Essa divisão facilita compreender e testar as regras separadamente da apresentação.

```
navio = Navio(3, "Cruzador")
# Cria um objeto com tamanho 3 e nome Cruzador.
```

Listas guardam sequências; tuplas representam coordenadas; dicionários associam coordenadas a resultados. A grade é uma lista de listas. Internamente a contagem começa em zero: C2 corresponde à linha 1 e coluna 2.

2 Arquivo principal batalha_naval py

Linhas 1 a 5 Documentação do módulo

```
001  """Ponto de entrada da Batalha Naval.
002
003  Execute este arquivo para abrir o menu do jogo. As classes principais são
004  reexportadas para manter compatibilidade com testes e comandos antigos.
005  """
```

A string entre aspas triplas é a docstring do arquivo. Documenta o ponto de entrada e a reexportação para compatibilidade. Ela não é impressa automaticamente.

Linhas 7 a 7 Importação do menu

```
007  from interface import menu
```

from interface import menu traz a função menu de interface.py para este módulo. Importar o módulo também executa suas instruções de nível superior, incluindo as configurações de saída.

Linhas 8 a 16 Reexportação das regras

```
008  from jogo import (
009      Computador,
010      Jogador,
011      Navio,
012      Tabuleiro,
013      frota_padrao,
014      ler_coordenada,
015      posicionar_frota_automaticamente,
016  )
```

Importa Computador, Jogador, Navio, Tabuleiro e as três funções auxiliares. Os parênteses permitem quebrar a importação em várias linhas. Assim, from batalha_naval import Navio continua possível, embora a definição esteja em jogo.py.

Linhas 18 a 26 Lista de nomes exportados

```
018  __all__ = [
019      "Computador",
020      "Jogador",
021      "Navio",
022      "Tabuleiro",
023      "frota_padrao",
024      "ler_coordenada",
025      "posicionar_frota_automaticamente",
026  ]
```

__all__ lista os nomes usados em from batalha_naval import *. As strings nomeiam objetos já importados; não os executam. A lista não impede importações explícitas de outros nomes.

Linhas 29 a 30 Execução direta

```
029  if __name__ == "__main__":
030      menu()
```

Quando o arquivo é executado diretamente, __name__ recebe "__main__" e menu() é chamado. Quando importado, o nome do módulo é diferente e o menu não abre involuntariamente. Execute com python batalha_naval.py.

3 Regras do jogo no arquivo jogo py

Linhas 1 a 5 Responsabilidade do módulo

```
001  """Regras e modelos da Batalha Naval.
002
003  Este módulo não exibe menus nem lê input. Ele concentra as regras para que
004  elas possam ser testadas sem depender do terminal.
005  """
```

A docstring informa que as regras e os modelos ficam separados da interface. Isso permite testar ataques e posicionamentos sem interação pelo terminal.

Linhas 7 a 7 Aleatoriedade

```
007  import random
```

Importa random, da biblioteca padrão, usado para escolher tiros e embaralhar tentativas.

Linhas 10 a 11 Classe Navio

```
010  class Navio:
011      """Representa um navio e as casas que ele ocupa e recebeu tiros."""
```

Define o modelo de um navio. A docstring descreve que ele guarda as casas ocupadas e atingidas.

Linhas 13 a 15 Inicialização e validação do navio

```
013      def __init__(self, tamanho, nome):
014          if type(tamanho) is not int or tamanho < 1:
015              raise ValueError("O tamanho do navio deve ser um inteiro positivo.")
```

__init__ recebe a instância em self, além de tamanho e nome. A condição rejeita valores cujo tipo exato não seja int ou sejam menores que 1. raise ValueError sinaliza o valor inválido. O teste type(...) is int também rejeita booleanos.

Linhas 16 a 19 Estado do navio

```
016          self.tamanho = tamanho
017          self.nome = nome
018          self.posicoes = []
019          self.acertos = {}
```

Guarda tamanho e nome nos atributos. posicoes começa como lista vazia e receberá tuplas (linha, coluna). acertos começa como dicionário vazio e relacionará coordenadas atingidas a True. Cada instância possui seus próprios dados.

Linhas 21 a 22 Verificar afundamento

```
021      def esta_afundado(self):
022          return len(self.acertos) == self.tamanho
```

len(self.acertos) conta casas atingidas. A comparação com self.tamanho produz True ou False e return devolve esse resultado. Pressupõe que apenas acertos válidos sejam registrados pelas regras.

Linhas 25 a 26 Classe Tabuleiro

```
025  class Tabuleiro:
026      """Armazena navios, tiros e regras de posicionamento do tabuleiro."""
```

Reúne a matriz, a frota e os tiros recebidos. Centraliza a validação para evitar regras duplicadas na interface.

Linhas 28 a 31 Tamanho do tabuleiro

```
028      def __init__(self, tamanho=5):
029          if type(tamanho) is not int or not 1 <= tamanho <= 26:
030              raise ValueError("O tabuleiro deve ter entre 1 e 26 casas por lado.")
031          self.tamanho = tamanho
```

O parâmetro tamanho=5 permite criar Tabuleiro() sem passar argumento. Exige inteiro entre 1 e 26 e guarda o valor. O limite 26 corresponde às letras de A a Z usadas na apresentação.

Linhas 32 a 34 Matriz e registros

```
032          self.grade = [[None for _ in range(tamanho)] for _ in range(tamanho)]
033          self.navios = []
034          self.tiros = {}
```

A compreensão interna cria uma linha com tamanho casas None; a externa cria as linhas independentes. range(tamanho) percorre de zero a tamanho menos um e _ indica valor não usado. navios é a lista de objetos posicionados; tiros é o dicionário dos ataques recebidos.

Linhas 36 a 40 Validar coordenada

```
036      def validar_coordenada(self, linha, coluna):
037          if type(linha) is not int or type(coluna) is not int:
038              raise ValueError("Linha e coluna devem ser números inteiros.")
039          if not (0 <= linha < self.tamanho and 0 <= coluna < self.tamanho):
040              raise ValueError("Coordenada fora do tabuleiro.")
```

Exige linha e coluna do tipo int. Em seguida exige 0 <= índice < tamanho para ambos. and combina as duas condições; not detecta o caso inválido. Quando válida, a função termina sem return explícito e devolve None: seu objetivo é rejeitar coordenadas inadequadas.

Linhas 42 a 46 Condições para posicionar

```
042      def posicionar_navio(self, navio, linha, coluna, orientacao):
043          if navio.posicoes:
044              raise ValueError("Este navio já foi posicionado.")
045          if orientacao not in ("H", "V"):
046              raise ValueError("Use H para horizontal ou V para vertical.")
```

O método recebe navio, casa inicial e orientação. Uma lista navio.posicoes não vazia é verdadeira e indica objeto já colocado. orientacao not in ("H", "V") rejeita outras orientações. Os erros interrompem a tentativa antes de alterar a grade.

Linhas 48 a 49 Preparar validação de todas as casas

```
048          posicoes = []
049          for deslocamento in range(navio.tamanho):
```

Cria uma lista temporária e percorre deslocamentos de zero a navio.tamanho menos um. Validar antes de gravar evita deixar parte de um navio no tabuleiro se a última casa for inválida.

Linhas 50 a 51 Calcular horizontal e vertical

```
050              nova_linha = linha + (deslocamento if orientacao == "V" else 0)
051              nova_coluna = coluna + (deslocamento if orientacao == "H" else 0)
```

Na vertical, a linha aumenta pelo deslocamento e a coluna permanece. Na horizontal, ocorre o contrário. A expressão valor if condição else outro escolhe o acréscimo. Um navio horizontal de tamanho 3 em (1, 1) ocupa (1, 1), (1, 2), (1, 3).

Linhas 52 a 55 Validar cada casa

```
052              self.validar_coordenada(nova_linha, nova_coluna)
053              if self.grade[nova_linha][nova_coluna] is not None:
054                  raise ValueError("Um navio não pode sobrepor outro.")
055              posicoes.append((nova_linha, nova_coluna))
```

Chama validar_coordenada para impedir saída da grade. Se a casa não é None, já existe navio e ocorre ValueError. append adiciona a tupla calculada à lista temporária quando ela é válida.

Linhas 57 a 60 Efetivar posicionamento

```
057          for nova_linha, nova_coluna in posicoes:
058              self.grade[nova_linha][nova_coluna] = navio
```

```
059          navio.posicoes = posicoes
060          self.navios.append(navio)
```
Percorre as posições aprovadas e grava a referência ao mesmo objeto Navio em cada casa. Depois atribui a lista ao navio e o adiciona à frota. Assim, tiros em casas diferentes atualizam o mesmo objeto.

Linhas 62 a 66 Validar tiro e impedir repetição

```
062      def receber_tiro(self, linha, coluna):
063          self.validar_coordenada(linha, coluna)
064          coordenada = (linha, coluna)
065          if coordenada in self.tiros:
066              raise ValueError("Esta coordenada já recebeu tiro.")
```
Valida os índices, cria a tupla coordenada e verifica sua presença entre as chaves de self.tiros. Se já existir, lança erro para impedir repetir o ataque.

Linhas 68 a 72 Registrar resultado do tiro

```
068          alvo = self.grade[linha][coluna]
069          self.tiros[coordenada] = alvo
070          if alvo is not None:
071              alvo.acertos[coordenada] = True
072          return alvo
```
Obtém o conteúdo da casa: Navio ou None. Registra o tiro mesmo se for água. Se houver navio, registra True em seus acertos. return alvo devolve o objeto atingido ou None para a interface interpretar.

Linhas 74 a 75 Contar afundados

```
074      def quantidade_navios_afundados(self):
075          return sum(navio.esta_afundado() for navio in self.navios)
```
A expressão geradora consulta esta_afundado em cada navio. sum soma os booleanos: True vale 1 e False vale 0. Não precisa montar uma lista intermediária.

Linhas 78 a 84 Classe Jogador e seus dados

```
078  class Jogador:
079      """Jogador humano ou base para o jogador controlado pelo computador."""
080
081      def __init__(self, nome):
082          self.nome = nome
083          self.tabuleiro = Tabuleiro()
084          self.tiros = {}
```
Define o jogador humano e a base do computador. O inicializador guarda nome, cria um Tabuleiro padrão e um dicionário de ataques realizados. jogador.tiros contém os ataques feitos; jogador.tabuleiro.tiros contém os ataques recebidos.

Linhas 86 a 91 Realizar ataque

```
086      def atacar(self, adversario, linha, coluna):
087          if adversario is self:
088              raise ValueError("Um jogador não pode atacar o próprio tabuleiro.")
089          resultado = adversario.tabuleiro.receber_tiro(linha, coluna)
090          self.tiros[(linha, coluna)] = resultado
091          return resultado
```
is compara identidade e impede atacar a própria instância. O ataque é encaminhado ao tabuleiro adversário, que valida e registra. O resultado é guardado nos tiros do atacante e devolvido. Se receber_tiro lançar erro, as instruções seguintes não executam.

Linhas 93 a 96 Verificar derrota

```
093      def verificar_derrota(self):
094          return bool(self.tabuleiro.navios) and all(
095              navio.esta_afundado() for navio in self.tabuleiro.navios
096          )
```

bool(navios) exige que exista frota. all exige que todos os navios estejam afundados. A primeira condição é importante porque all sobre uma sequência vazia retorna True. and usa curto-circuito e evita avaliar a segunda parte quando não há frota.

Linhas 99 a 100 Herança do computador

```
099  class Computador(Jogador):
100      """Computador que escolhe tiros aleatórios ou persegue um acerto."""
```

Computador(Jogador) herda a estrutura de jogador, incluindo atacar e verificar_derrota, e acrescenta decisões automáticas.

Linhas 102 a 105 Inicializar computador

```
102      def __init__(self, dificuldade):
103          super().__init__("Computador")
104          self.dificuldade = dificuldade
105          self.alvos_pendentes = []
```

super().__init__("Computador") executa a inicialização da classe base. Em seguida guarda dificuldade e inicia a lista de alvos prioritários vazia.

Linhas 107 a 113 Descobrir tiros disponíveis

```
107      def escolher_tiro(self):
108          disponiveis = [
109              (linha, coluna)
110              for linha in range(self.tabuleiro.tamanho)
111              for coluna in range(self.tabuleiro.tamanho)
112              if (linha, coluna) not in self.tiros
113          ]
```

A compreensão de lista percorre linhas e colunas e inclui apenas coordenadas ausentes dos tiros feitos. Usa o tamanho do próprio tabuleiro, pressupondo tabuleiros iguais, como ocorre na partida atual.

Linhas 114 a 115 Escolha aleatória

```
114          if self.dificuldade == "fácil" or not self.alvos_pendentes:
115              return random.choice(disponiveis)
```

Se está no fácil ou a fila de prioridades está vazia, random.choice escolhe uma coordenada disponível. choice exige uma sequência não vazia. O fluxo normal encerra a partida antes de precisar atirar com todas as casas esgotadas.

Linhas 117 a 121 Escolha por prioridade

```
117          while self.alvos_pendentes:
118              alvo = self.alvos_pendentes.pop(0)
119              if alvo in disponiveis:
120                  return alvo
121          return random.choice(disponiveis)
```

Enquanto houver alvos, pop(0) remove e devolve o primeiro. Se ainda está disponível, return o escolhe. Os alvos já atacados são descartados. Se a fila esgotar, escolhe uma casa aleatória.

Linhas 123 a 128 Aprender com o resultado

```
123      def registrar_resultado(self, linha, coluna, resultado):
124          if resultado is None or self.dificuldade == "fácil":
125              return
126          if resultado.esta_afundado():
127              self.alvos_pendentes.clear()
128              return
```

Se foi água ou está no fácil, encerra com return. Se o navio afundou, clear apaga a fila e o método termina. As prioridades só são adicionadas após um acerto em navio ainda vivo.

Linhas 130 a 136 Gerar vizinhos do acerto

```
130         # Depois de acertar, adiciona as casas vizinhas como próximos alvos.
131         for vizinho in (
132             (linha - 1, coluna),
133             (linha + 1, coluna),
134             (linha, coluna - 1),
135             (linha, coluna + 1),
136         ):
```

O comentário explica a intenção. O for percorre as casas acima, abaixo, à esquerda e à direita. Não inclui diagonais porque os navios são alinhados horizontal ou verticalmente.

Linhas 137 a 143 Filtrar vizinhos

```
137             if (
138                 0 <= vizinho[0] < self.tabuleiro.tamanho
139                 and 0 <= vizinho[1] < self.tabuleiro.tamanho
140                 and vizinho not in self.tiros
141                 and vizinho not in self.alvos_pendentes
142             ):
143                 self.alvos_pendentes.append(vizinho)
```

Exige que linha e coluna estejam dentro da grade, que a coordenada ainda não tenha sido atacada e que não esteja na fila. Só então append acrescenta o vizinho. Evita alvos inválidos e duplicados.

Linhas 146 a 149 Normalizar coordenada textual

```
146 def ler_coordenada(texto, tamanho=5):
147     """Converte 'A1' em índices internos (0, 0)."""
148     texto = texto.strip().upper()
149     letras = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"[:tamanho]
```

A função recebe texto e tamanho padrão 5. strip remove espaços das pontas e upper converte letras para maiúsculas. O fatiamento seleciona apenas as letras disponíveis para o tabuleiro.

Linhas 150 a 151 Validar formato textual

```
150     if len(texto) < 2 or texto[0] not in letras or not texto[1:].isdigit():
151         raise ValueError("Use uma coordenada como A1 ou E5.")
```

Exige pelo menos dois caracteres, uma letra válida no começo e dígitos no restante. O or usa curto-circuito: em texto vazio, a condição de comprimento já é verdadeira e não acessa texto[0].

Linhas 152 a 156 Converter para índices

```
152     linha = int(texto[1:]) - 1
153     coluna = letras.index(texto[0])
154     if not 0 <= linha < tamanho:
155         raise ValueError("Coordenada fora do tabuleiro.")
156     return linha, coluna
```

int(texto[1:]) converte a parte numérica e subtrai 1. letras.index encontra a coluna. Valida a linha e devolve o par. C2 vira (1, 2): letra é coluna e número é linha. A validação da letra já ocorreu antes.

Linhas 159 a 161 Definir frota

```
159 def frota_padrao():
160     """Retorna os navios usados em cada partida."""
161     return [(3, "Cruzador"), (2, "Destroyer"), (1, "Submarino")]
```

Retorna a lista de tuplas com tamanho e nome: Cruzador 3, Destroyer 2 e Submarino 1. São dados para criar novos objetos; cada jogador recebe seus próprios navios.

Linhas 164 a 172 Preparar posicionamento automático

```
164 def posicionar_frota_automaticamente(jogador):
165     """Posiciona a frota embaralhando todas as tentativas possíveis."""
166     for tamanho, nome in frota_padrao():
167         candidatos = [
168             (linha, coluna, orientacao)
```

```
169             for linha in range(jogador.tabuleiro.tamanho)
170             for coluna in range(jogador.tabuleiro.tamanho)
171             for orientacao in ("H", "V")
172         ]
```

Para cada tamanho e nome desempacotados da frota, gera todos os candidatos de linha, coluna e orientação. As três repetições da compreensão produzem combinações.

Linhas 173 a 179 Testar candidatos embaralhados

```
173         random.shuffle(candidatos)
174         for linha, coluna, orientacao in candidatos:
175             try:
176                 jogador.tabuleiro.posicionar_navio(
177                     Navio(tamanho, nome), linha, coluna, orientacao
178                 )
179                 break
```

shuffle altera a ordem da lista. Para cada candidato, try tenta posicionar um novo Navio. Se funcionar, break encerra o laço interno e segue para o próximo navio da frota.

Linhas 180 a 183 Tratar tentativas inválidas

```
180             except ValueError:
181                 continue
182         else:
183             raise RuntimeError("Não foi possível posicionar a frota.")
```

except ValueError captura saídas da grade ou sobreposições e continue tenta o próximo candidato. O else pertence ao for e executa somente se ele terminar sem break: nenhuma tentativa funcionou. RuntimeError comunica a falha. O algoritmo não volta para reorganizar navios anteriores.

4 Interface no arquivo interface py

Linhas 1 a 6 Documentação e bibliotecas

```
001  """Interface de terminal da Batalha Naval."""
002
003  import json
004  import sys
005  from datetime import datetime
006  from pathlib import Path
```

A docstring apresenta o módulo. json converte o histórico, sys permite configurar saídas, datetime fornece data e hora e Path representa caminhos de arquivos.

Linhas 8 a 16 Importar as regras

```
008  from jogo import (
009      Computador,
010      Jogador,
011      Navio,
012      Tabuleiro,
013      frota_padrao,
014      ler_coordenada,
015      posicionar_frota_automaticamente,
016  )
```

Traz as classes e funções de jogo.py. A interface pode pedir ações aos modelos sem reimplementar suas validações.

Linhas 19 a 21 Configurar codificação

```
019  if hasattr(sys.stdout, "reconfigure"):
020      sys.stdout.reconfigure(encoding="utf-8")
021      sys.stderr.reconfigure(encoding="utf-8")
```

hasattr verifica se stdout possui reconfigure. Configura saída normal e de erros em UTF-8 para acentos e símbolos. A verificação é feita apenas em stdout; pressupõe stderr também compatível.

Linhas 24 a 30 Cores e negrito

```
024  RESET = "\033[0m"
025  AZUL = "\033[94m"
026  CIANO = "\033[96m"
027  VERDE = "\033[92m"
028  VERMELHO = "\033[91m"
029  AMARELO = "\033[93m"
030  NEGRITO = "\033[1m"
```

Define strings com sequências ANSI. RESET restaura, AZUL 94, CIANO 96, VERDE 92, VERMELHO 91, AMARELO 93 e NEGRITO 1 formatam o terminal compatível. Maiúsculas são convenção de constantes, não proibição de alteração.

Linhas 31 a 31 Caminho do histórico

```
031  HISTORICO = Path(__file__).with_name("historico.json")
```

Path(__file__) representa o arquivo atual. with_name troca seu nome final por historico.json, mantendo a pasta. O local independe da pasta de onde o usuário executa o comando.

Linhas 34 a 35 Colorir texto

```
034  def colorir(texto, cor):
035      return f"{cor}{texto}{RESET}"
```

A f-string concatena código da cor, texto e RESET. Retorna uma string; não imprime por conta própria. O reset impede que a cor se propague às próximas mensagens.

Linhas 38 a 39 Limpar terminal

```
038  def limpar_tela():
039      print("\033[2J\033[H", end="")
```

A sequência ANSI 2J limpa a tela e H leva o cursor ao início. end="" impede a quebra de linha adicional do print. Depende de suporte ANSI.

Linhas 42 a 43 Pausar

```
042  def pausar(mensagem="Pressione ENTER para continuar..."):
043      input(f"\n{mensagem}")
```

O argumento tem uma mensagem padrão que pode ser substituída. input aguarda Enter. A resposta é descartada porque a finalidade é pausar. A sequência \n adiciona uma linha antes da mensagem.

Linhas 46 a 49 Preparar desenho do tabuleiro

```
046  def exibir_tabuleiro(tabuleiro, revelar_navios=False):
047      """Monta a grade para a tela sem misturar apresentação com as regras."""
048      letras = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"[:tabuleiro.tamanho]
049      linhas = [f"    {' '.join(letras)}"]
```

revelar_navios=False oculta navios intactos por padrão. Seleciona letras e cria o cabeçalho. join une as letras com espaços. linhas acumula o desenho que será devolvido.

Linhas 50 a 54 Percorrer as casas

```
050      for linha in range(tabuleiro.tamanho):
051          valores = []
052          for coluna in range(tabuleiro.tamanho):
053              coordenada = (linha, coluna)
054              navio = tabuleiro.grade[linha][coluna]
```

O laço externo percorre linhas e reinicia valores. O interno percorre colunas, cria a coordenada e consulta a matriz para obter Navio ou None.

Linhas 55 a 60 Escolher símbolos

```
055              if coordenada in tabuleiro.tiros:
056                  simbolo = colorir("X", VERMELHO) if navio else colorir("O", AZUL)
057              elif revelar_navios and navio:
058                  simbolo = colorir("N", VERDE)
059              else:
060                  simbolo = colorir("~", CIANO)
```

Se já recebeu tiro, usa X vermelho quando há navio ou O azul se é água. Caso contrário, mostra N verde se revelar_navios e navio forem verdadeiros. Nos demais casos usa ~ ciano. A prioridade faz casas atingidas exibirem o resultado do tiro.

Linhas 61 a 63 Montar o texto final

```
061              valores.append(simbolo)
062          linhas.append(f"{linha + 1:>2}  {' '.join(valores)}")
063      return "\n".join(linhas)
```

append guarda cada símbolo. Ao terminar uma linha, usa linha+1 para numeração humana e :>2 para alinhamento à direita em largura mínima 2. join une as casas por espaços e as linhas por quebras. A função devolve texto, sem imprimi-lo.

Linhas 66 a 70 Iniciar posicionamento manual

```
066  def posicionar_frota_manual(jogador):
067      """Pede ao jogador a casa inicial e a orientação de cada navio."""
068      print("\nPosicione sua frota. Digite 'auto' para posicionar automaticamente.")
069      for tamanho, nome in frota_padrao():
070          while True:
```

Informa a alternativa auto, percorre cada navio da frota e usa while True para insistir até o posicionamento dar certo.

Linhas 71 a 74 Solicitar posição

```
071             limpar_tela()
072             print(colorir(f"Posicionando {nome} (tamanho {tamanho})", NEGRITO + CIANO))
073             print(exibir_tabuleiro(jogador.tabuleiro, revelar_navios=True))
074             texto = input("\nCoordenada inicial (ex.: A1): ").strip()
```

Limpa a tela, exibe nome e tamanho com negrito e cor, revela o próprio tabuleiro e lê a casa inicial removendo espaços das extremidades.

Linhas 75 a 78 Alternativa automática

```
075             if texto.lower() == "auto":
076                 jogador.tabuleiro = Tabuleiro()
077                 posicionar_frota_automaticamente(jogador)
078                 return
```

lower permite reconhecer AUTO, Auto ou auto. Substitui o tabuleiro por um novo, posiciona toda a frota automaticamente e retorna. Portanto, descarta posicionamentos manuais anteriores.

Linhas 79 a 85 Tentar colocar navio

```
079             try:
080                 linha, coluna = ler_coordenada(texto)
081                 orientacao = input("Orientação (H = horizontal, V = vertical): ").strip().upper()
082                 jogador.tabuleiro.posicionar_navio(
083                     Navio(tamanho, nome), linha, coluna, orientacao
084                 )
085                 break
```

try converte a coordenada e recebe orientação com strip e upper. Cria o objeto Navio e solicita posicionamento ao tabuleiro. break sai do while após sucesso, avançando para o próximo navio.

Linhas 86 a 88 Mostrar erro e repetir

```
086             except ValueError as erro:
087                 print(colorir(f"\nAtenção: {erro}", AMARELO))
088                 pausar()
```

Captura ValueError na variável erro, imprime a mensagem em amarelo e pausa. Ao acabar o bloco, o while recomeça para o mesmo navio.

Linhas 91 a 96 Cabeçalho da rodada

```
091 def mostrar_status(humano, computador, rodada):
092     print(colorir("=" * 61, CIANO))
093     print(colorir("                 BATALHA NAVAL", NEGRITO + CIANO))
094     print(colorir(f"                         RODADA {rodada}", CIANO))
095     print(colorir("=" * 61, CIANO))
096     print(f"{colorir(humano.nome, NEGRITO)}                 {colorir('COMPUTADOR', NEGRITO)}")
```

Imprime bordas repetindo caracteres com *, título, número da rodada e nomes. As f-strings inserem os valores e colorir aplica formatação.

Linhas 97 a 100 Desenhar os dois lados

```
097     print(exibir_tabuleiro(humano.tabuleiro, revelar_navios=True))
098     print()
099     print(exibir_tabuleiro(computador.tabuleiro))
100     print()
```

Mostra o tabuleiro humano revelado, imprime linha vazia e mostra o computador com a opção padrão que oculta os navios intactos. Os tabuleiros são impressos um abaixo do outro.

Linhas 101 a 104 Mostrar contagem

```
    print(
        f"Seus navios afundados: {humano.tabuleiro.quantidade_navios_afundados()}/3"
        f"    Inimigos afundados: {computador.tabuleiro.quantidade_navios_afundados()}/3"
    )
```

Chama quantidade_navios_afundados para cada lado. Strings consecutivas dentro dos parênteses são concatenadas. O denominador /3 está fixo e precisa acompanhar mudanças na frota.

Linhas 105 a 108 Legenda

```
    print(colorir("—" * 61, CIANO))
    print(f"{colorir('N', VERDE)} navio  {colorir('X', VERMELHO)} acerto  "
          f"{colorir('O', AZUL)} água  {colorir('~', CIANO)} desconhecido")
    print(colorir("—" * 61, CIANO))
```

Imprime separadores e associa N a navio, X a acerto, O a água e ~ a desconhecido. As cores reforçam os símbolos.

Linhas 111 a 116 Mensagem de tiro

```
def mensagem_tiro(jogador, resultado):
    if resultado is None:
        return colorir(f"{jogador} acertou a água.", AZUL)
    if resultado.esta_afundado():
        return colorir(f"{jogador} afundou o {resultado.nome}!", VERMELHO)
    return colorir(f"{jogador} acertou o {resultado.nome}!", AMARELO)
```

Se resultado é None, retorna água. Se o objeto está afundado, retorna o nome do navio afundado. Caso contrário retorna acerto. Cada return encerra a função, dispensando else após os testes.

Linhas 119 a 127 Mostrar regras

```
def mostrar_regras():
    limpar_tela()
    print(colorir("REGRAS", NEGRITO + CIANO))
    print("\n1. Posicione um Cruzador, um Destroyer e um Submarino.")
    print("2. Ataque informando coordenadas como A1, B3 ou E5.")
    print("3. X representa acerto, O representa água e ~ é desconhecido.")
    print("4. O primeiro jogador a afundar toda a frota vence.")
    print("5. Use 'sair' durante a partida para voltar ao menu.")
    pausar()
```

Limpa a tela, exibe título e cinco instruções e pausa. A função apresenta as regras; sua aplicação acontece nos modelos e no fluxo da partida.

Linhas 130 a 136 Carregar histórico

```
def carregar_historico():
    if not HISTORICO.exists():
        return []
    try:
        return json.loads(HISTORICO.read_text(encoding="utf-8"))
    except (json.JSONDecodeError, OSError):
        return []
```

Se o caminho não existe, retorna lista vazia. Caso exista, read_text lê UTF-8 e json.loads transforma JSON em estruturas Python. Se ocorrer JSONDecodeError ou OSError, retorna lista vazia. Não valida a estrutura de um JSON sintaticamente válido.

Linhas 139 a 147 Criar registro da partida

```
def salvar_resultado(nome, resultado, rodadas, dificuldade):
    historico = carregar_historico()
    historico.append({
        "jogador": nome,
        "resultado": resultado,
        "rodadas": rodadas,
        "dificuldade": dificuldade,
        "data": datetime.now().strftime("%d/%m/%Y %H:%M"),
```

```
147     })
```
Carrega a lista e acrescenta um dicionário com jogador, resultado, rodadas, dificuldade e data. datetime.now obtém hora local e strftime usa %d dia, %m mês, %Y ano, %H hora 24h e %M minuto.

Linhas 148 a 151 Gravar histórico

```
148     HISTORICO.write_text(
149         json.dumps(historico[-20:], ensure_ascii=False, indent=2),
150         encoding="utf-8",
151     )
```
Seleciona historico[-20:] para manter os últimos 20 registros. dumps converte para JSON, ensure_ascii=False preserva acentos legíveis e indent=2 organiza o texto. write_text grava UTF-8 substituindo o arquivo. Erros de escrita não são capturados aqui.

Linhas 154 a 160 Preparar consulta do histórico

```
154 def mostrar_historico():
155     limpar_tela()
156     historico = carregar_historico()
157     print(colorir("HISTÓRICO DE PARTIDAS", NEGRITO + CIANO))
158     if not historico:
159         print("\nNenhuma partida registrada.")
160     else:
```
Limpa tela, carrega dados e imprime título. Se a lista está vazia, informa que não há partidas; caso contrário entra na exibição dos registros.

Linhas 161 a 167 Listar partidas recentes

```
161         for partida in reversed(historico):
162             print(
163                 f"\n{partida['data']} | {partida['jogador']} | "
164                 f"{partida['resultado']} | {partida['rodadas']} rodadas | "
165                 f"{partida['dificuldade']}"
166             )
167     pausar()
```
reversed percorre do último para o primeiro. Acessa campos do dicionário pelas chaves entre colchetes e os insere nas f-strings. Exibe data, nome, resultado, rodadas e dificuldade. Ao final pausa.

Linhas 170 a 176 Escolher dificuldade

```
170 def escolher_dificuldade():
171     opcoes = {"1": "fácil", "2": "médio", "3": "difícil"}
172     while True:
173         escolha = input("\nDificuldade: 1-Fácil  2-Médio  3-Difícil: ").strip()
174         if escolha in opcoes:
175             return opcoes[escolha]
176         print(colorir("Escolha 1, 2 ou 3.", AMARELO))
```
O dicionário mapeia strings 1, 2 e 3 para nomes. while repete a pergunta; se a escolha está nas chaves, devolve o valor associado. Caso inválida, imprime aviso e tenta de novo. input retorna string, por isso as chaves são textuais.

Linhas 179 a 185 Criar partida

```
179 def jogar():
180     limpar_tela()
181     print(colorir("BATALHA NAVAL", NEGRITO + CIANO))
182     nome = input("\nComo devemos chamar você? ").strip() or "Jogador"
183     dificuldade = escolher_dificuldade()
184     humano = Jogador(nome)
185     computador = Computador(dificuldade)
```
Limpa a tela e pede nome. strip remove espaços; or usa Jogador se o texto ficar vazio. Obtém dificuldade e cria humano e computador com estados independentes.

Linhas 186 a 189 Preparar frota e rodada

```
186      posicionar_frota_manual(humano)
187      posicionar_frota_automaticamente(computador)
188      pausar("Frota pronta! Pressione ENTER para começar...")
189      rodada = 1
```

Posiciona a frota humana manualmente ou pela opção auto, a do computador automaticamente, pausa e inicia rodada em 1.

Linhas 191 a 198 Iniciar turno ou desistir

```
191      while True:
192          limpar_tela()
193          mostrar_status(humano, computador, rodada)
194          texto = input(f"\n{nome}, escolha um alvo (A1-E5) ou 'sair': ").strip()
195          if texto.lower() == "sair":
196              salvar_resultado(nome, "Desistência", rodada, dificuldade)
197              print(colorir("\nPartida encerrada.", AMARELO))
198              return
```

O laço principal limpa, exibe status e lê o ataque. Se a entrada for sair, salva Desistência, imprime aviso e retorna ao chamador, que é o menu.

Linhas 199 a 205 Validar e executar ataque humano

```
199          try:
200              linha, coluna = ler_coordenada(texto)
201              resultado = humano.atacar(computador, linha, coluna)
202          except ValueError as erro:
203              print(colorir(f"\nAtenção: {erro}", AMARELO))
204              pausar()
205              continue
```

Converte a coordenada e chama atacar. Se houver ValueError por entrada inválida ou tiro repetido, mostra aviso, pausa e continue reinicia a rodada. O computador não joga nesse caso e o contador não aumenta.

Linhas 207 a 212 Verificar vitória humana

```
207          print(f"\n{mensagem_tiro('Você', resultado)}")
208          if computador.verificar_derrota():
209              salvar_resultado(nome, "Vitória", rodada, dificuldade)
210              print(colorir("\nPARABÉNS! Você venceu!", VERDE + NEGRITO))
211              pausar()
212              return
```

Mostra o resultado do tiro e verifica derrota do computador antes de permitir sua vez. Se perdeu, salva Vitória, mostra parabéns, pausa e retorna. Evita uma resposta do adversário já derrotado.

Linhas 214 a 218 Executar turno do computador

```
214          linha, coluna = computador.escolher_tiro()
215          resultado = computador.atacar(humano, linha, coluna)
216          computador.registrar_resultado(linha, coluna, resultado)
217          print(f"Computador escolheu {chr(65 + coluna)}{linha + 1}.")
218          print(mensagem_tiro("O computador", resultado))
```

Escolhe coordenada, ataca e registra resultado na estratégia. chr(65+coluna) converte índice em letra A, B etc.; linha+1 converte o número. Mostra a coordenada e a mensagem do tiro.

Linhas 219 a 225 Verificar derrota e avançar

```
219          if humano.verificar_derrota():
220              salvar_resultado(nome, "Derrota", rodada, dificuldade)
221              print(colorir("\nO computador venceu.", VERMELHO + NEGRITO))
222              pausar()
223              return
224          rodada += 1
225          pausar()
```

Se o humano perdeu, registra Derrota, mostra mensagem, pausa e retorna. Caso a partida continue, rodada += 1 soma uma rodada e pausar aguarda antes da próxima. Acertar não concede tiro extra neste código.

Linhas 228 a 238 Desenhar menu

```
228  def menu():
229      while True:
230          limpar_tela()
231          print(colorir("╔═══════════════════════════════════════╗", CIANO))
232          print(colorir("║          BATALHA NAVAL                ║", NEGRITO + CIANO))
233          print(colorir("╠═══════════════════════════════════════╣", CIANO))
234          print("║  1 - Jogar                            ║")
235          print("║  2 - Ver regras                       ║")
236          print("║  3 - Ver histórico                    ║")
237          print("║  4 - Sair                             ║")
238          print(colorir("╚═══════════════════════════════════════╝", CIANO))
```

Define a função e um while que repete até sair. Limpa a tela e imprime moldura, título e opções 1 Jogar, 2 Regras, 3 Histórico e 4 Sair.

Linhas 239 a 245 Encaminhar opções

```
239          escolha = input("\nEscolha uma opção: ").strip()
240          if escolha == "1":
241              jogar()
242          elif escolha == "2":
243              mostrar_regras()
244          elif escolha == "3":
245              mostrar_historico()
```

Lê a escolha e remove espaços. if e elif selecionam jogar, mostrar_regras ou mostrar_historico. Quando a função chamada termina, o laço apresenta o menu novamente.

Linhas 246 a 251 Encerrar ou rejeitar escolha

```
246          elif escolha == "4":
247              print("\nAté a próxima!")
248              return
249          else:
250              print(colorir("\nOpção inválida.", AMARELO))
251              pausar()
```

A opção 4 imprime despedida e retorna, encerrando o menu. O else mostra opção inválida e pausa, antes de repetir.

5 Glossário completo dos recursos usados

Aqui, comandos inclui palavras da linguagem, funções prontas, métodos, operadores e estruturas. Nomes criados pelo projeto não são comandos internos do Python.

Palavras da linguagem

Recurso	Explicação
import	Importa um módulo. Exemplo: import random.
from ... import ...	Importa nomes específicos de outro módulo.
class	Define uma classe.
def	Define função ou método.
if	Executa um bloco se a condição for verdadeira.
elif	Testa outra condição após as anteriores falharem.
else	Alternativa do if; após for, executa quando o laço termina sem break.
for	Percorre elementos de uma sequência ou iterável.
while	Repete enquanto a condição for verdadeira.
return	Encerra a função atual; pode devolver um resultado. Sem valor, devolve None.
break	Sai apenas do laço mais próximo.
continue	Começa a próxima repetição do laço mais próximo.
try	Delimita operações cujos erros podem ser tratados.
except	Captura os tipos de exceção especificados.
as	Associa o erro capturado a um nome, como erro.
raise	Lança uma exceção.
and	Em condições exige ambas verdadeiras; retorna um dos operandos e usa curto-circuito.
or	Retorna o primeiro operando verdadeiro ou o último; usa curto-circuito.
not	Nega o valor lógico.
in e not in	Verificam presença ou ausência; em dicionários consultam chaves. in também integra a sintaxe do for.
is e is not	Comparam identidade de objetos. Usados para None e para impedir autoataque.

Valores e funções internas

Recurso	Explicação
True e False	Valores booleanos verdadeiro e falso. Em soma, correspondem a 1 e 0.

Recurso	Explicação
None	Ausência de objeto ou resultado: uma casa sem navio.
print()	Mostra dados na saída padrão. end="" elimina a quebra de linha final.
input()	Lê uma linha digitada e retorna string sem a quebra final.
type()	Obtém o tipo de um objeto.
len()	Conta elementos de uma coleção ou caracteres de uma string.
range()	Produz sequência de inteiros, excluindo o limite final.
sum()	Soma valores de um iterável.
bool()	Converte para booleano; coleções vazias são falsas.
all()	True se todos os elementos forem verdadeiros; também é True para sequência vazia.
super()	Acessa comportamento da classe base conforme a ordem de herança.
int()	Converte valores compatíveis em inteiros. int também nomeia o tipo.
hasattr()	Verifica a existência de um atributo.
reversed()	Permite percorrer uma sequência em ordem inversa.
chr()	Transforma código Unicode em caractere; chr(65) é A.

Métodos de strings e listas

Recurso	Explicação
strip()	Remove espaços e caracteres de espaço das extremidades.
upper() e lower()	Convertem letras para maiúsculas ou minúsculas.
isdigit()	True para string não vazia composta por caracteres considerados dígitos.
index()	Retorna a primeira posição do valor procurado; lança ValueError se ausente.
join()	Une strings usando a string à esquerda como separador.
append()	Adiciona um elemento ao final da lista.
pop(0)	Remove e devolve o primeiro elemento da lista.
clear()	Esvazia a coleção.

Bibliotecas e arquivos

Recurso	Explicação
random.choice()	Escolhe um elemento de uma sequência não vazia.
random.shuffle()	Embaralha a lista no próprio lugar; não retorna uma nova lista.

Recurso	Explicação
sys.stdout e sys.stderr	Objetos das saídas normal e de erros.
reconfigure()	Reconfigura um fluxo de texto compatível; encoding="utf-8" ajusta codificação.
datetime.now()	Obtém data e hora locais.
strftime()	Formata data e hora em texto.
Path()	Cria um objeto que representa um caminho.
with_name()	Produz caminho com nome final substituído.
exists()	Verifica se o caminho existe.
read_text()	Lê um arquivo como texto com a codificação indicada.
write_text()	Grava texto e substitui conteúdo existente.
json.loads()	Converte uma string JSON para objetos Python.
json.dumps()	Converte objetos Python para uma string JSON.
ensure_ascii=False	Mantém caracteres não ASCII legíveis no JSON produzido.
indent=2	Formata o JSON com recuo de dois espaços.

Exceções

Recurso	Explicação
ValueError	Valor inadequado: tamanho, coordenada, orientação ou ação inválida.
RuntimeError	Falha de operação sinalizada pelo posicionamento automático.
json.JSONDecodeError	Texto com sintaxe JSON inválida.
OSError	Erro de operação de sistema, como acesso a arquivo.

Operadores e símbolos

Recurso	Explicação
= e ==	= atribui; == compara igualdade de valores.
< e <=	Comparações menor que e menor ou igual. 0 <= linha < tamanho é comparação encadeada.
+ e -	Somam e subtraem números; + também concatena strings.
*	No projeto repete strings: "═" * 61. Em import *, seleciona os nomes exportados.
+=	Atualiza somando: rodada += 1 equivale aqui a rodada = rodada + 1.
[] e [a, b]	Lista vazia ou com elementos. Colchetes também acessam índices e chaves.

Recurso	Explicação
{} e {chave: valor}	Dicionário vazio ou com associações. {} não é conjunto vazio.
(linha, coluna)	Tupla de dois valores. Tuplas de inteiros podem ser chaves de dicionário.
objeto.atributo e objeto.metodo()	Ponto acessa atributo; parênteses realizam a chamada.
:	Inicia blocos e aparece em dicionários, fatias e formatação.
,	Separa argumentos, itens e variáveis; participa da criação de tuplas.
#	Comentário até o fim da linha.
Aspas simples e duplas	Delimitam strings. Aspas triplas permitem strings multilinha e docstrings.
Indentação	Define quais instruções pertencem a cada bloco. Não é apenas decoração.

Índices e construções

Recurso	Explicação
texto[0]	Primeiro caractere; índices começam em zero.
texto[1:]	Do segundo caractere até o fim.
letras[:tamanho]	Do início até antes do índice tamanho.
historico[-20:]	Até os últimos vinte elementos.
grade[linha][coluna]	Seleciona a lista da linha e depois sua casa.
Compreensão de lista	[expressão for ... if ...] cria lista por repetição e filtro.
Expressão geradora	Produz valores sob demanda, como o argumento de sum e all.
Desempacotamento	linha, coluna = (1, 2) distribui os dois valores.
Expressão condicional	a if condição else b escolhe um valor.
Curto-circuito	and e or podem parar antes de avaliar todos os operandos. Protege texto[0] na validação.
Strings consecutivas	Literais adjacentes dentro dos parênteses são concatenados automaticamente.

Formatação e nomes especiais

Recurso	Explicação
f"Texto {valor}"	F-string insere valores e expressões.
\n	Caractere de quebra de linha.
{linha + 1:>2}	Alinha à direita com largura mínima 2.
\033	Escape usado como início dos comandos ANSI.

Recurso	Explicação
ANSI 0m e 1m	Restaurar formatação e ativar negrito ou intensidade.
ANSI 94m 96m 92m 91m 93m	Azul, ciano, verde, vermelho e amarelo brilhantes.
ANSI 2J e H	Limpar a tela e mover o cursor para o início.
%d %m %Y %H %M	Dia, mês, ano de quatro dígitos, hora de 24h e minuto. %m é mês; %M é minuto.
self	Referência à instância no método; convenção de nome.
__init__	Inicializa a instância recém-criada.
__name__ e __main__	Identificam o módulo e a execução como programa principal.
__file__	Caminho do arquivo do módulo.
__all__	Controla os nomes exportados por import *.
_	Nome convencional para valor que não será utilizado.
Nomes em maiúsculas	Convenção para constantes, como RESET e HISTORICO.

Classes métodos e funções definidos pelo projeto

Os quatro modelos são Navio, Tabuleiro, Jogador e Computador. Todos os inicializadores e métodos são detalhados na parte 3. As funções de jogo.py são ler_coordenada, frota_padrao e posicionar_frota_automaticamente. As funções da interface são colorir, limpar_tela, pausar, exibir_tabuleiro, posicionar_frota_manual, mostrar_status, mensagem_tiro, mostrar_regras, carregar_historico, salvar_resultado, mostrar_historico, escolher_dificuldade, jogar e menu. Cada uma tem seu código e sua finalidade explicados na parte 4.

6 Caminho completo de um ataque

```
Entrada do jogador: C2
    ler_coordenada("C2") retorna (1, 2)
    humano.atacar(computador, 1, 2)
    computador.tabuleiro.receber_tiro(1, 2)
        valida limites e rejeita repetição
        consulta grade[1][2]
        registra o tiro
        atualiza acertos do Navio, se houver
        retorna Navio ou None
    atacante guarda o resultado
    mensagem_tiro explica água, acerto ou afundamento
    verificar_derrota consulta toda a frota
```

Se a casa contiver um Cruzador de tamanho 3 e esse for seu terceiro acerto, esta_afundado retorna True. Se os outros navios também estiverem afundados, verificar_derrota retorna True e jogar registra a vitória antes da vez do computador.

7 Preparação para explicar o trabalho

Roteiro de apresentação

Organizei o projeto em três módulos. O principal inicia o programa, o módulo de regras representa navios, tabuleiros e jogadores, e a interface recebe comandos e mostra os resultados.

Usei orientação a objetos porque cada elemento possui dados e comportamentos próprios. O navio conhece tamanho e acertos; o tabuleiro controla posições e tiros; o jogador ataca. O computador herda de Jogador e acrescenta decisões automáticas.

Usei uma matriz para as casas, tuplas para coordenadas, listas para sequências e dicionários para associar coordenadas aos resultados. As regras lançam exceções e a interface as captura para o jogador tentar novamente.

Limitações que você deve saber explicar

Médio e difícil têm o mesmo comportamento atual. Fácil atira aleatoriamente; os outros perseguem vizinhos após um acerto.

A interface usa tabuleiro padrão de 5 por 5, texto A1 a E5 e contagem /3. Alterar apenas o modelo não adapta toda a interface.

O posicionamento automático não reorganiza navios anteriores quando esgota as opções do navio atual.

A leitura trata erros de acesso e JSON inválido, mas não valida o formato dos dados de um JSON válido. A escrita não captura falhas de gravação.

A aparência depende de um terminal compatível com ANSI. O comando auto reinicia o posicionamento de toda a frota.

Os atributos são acessíveis diretamente. As validações dos métodos pressupõem que o restante do programa respeite seus registros internos.

Perguntas para revisão

Por que as coordenadas começam em zero?

Porque os índices das listas em Python começam em zero. A interface converte a entrada humana para esses índices.

Qual a diferença entre =, == e is?

= atribui; == compara valores; is compara identidade de objetos.

Qual a diferença entre return, break e continue?

return sai da função; break sai do laço mais próximo; continue passa à próxima repetição desse laço.

Por que validar todas as posições antes de gravar o navio?

Para uma tentativa inválida não deixar parte do navio posicionada.

Por que repetir a referência ao mesmo Navio na grade?

Para que tiros em casas diferentes atualizem o mesmo conjunto de acertos.

Por que existem dois dicionários de tiros?

Um guarda ataques realizados pelo jogador; outro guarda os ataques recebidos por seu tabuleiro.

Por que bool(navios) aparece antes de all?

Porque all de uma coleção vazia é True. A condição impede derrota sem frota.

O que Computador herda?

A inicialização básica e os métodos de Jogador, incluindo atacar e verificar_derrota.

Por que uma entrada inválida não perde a vez?

O except usa continue antes do ataque do computador e antes de incrementar a rodada.

Como persistem os resultados após fechar o jogo?

São gravados em historico.json, lidos na próxima execução.

O computador conhece a frota adversária para escolher tiros?

A escolha usa as coordenadas disponíveis e resultados anteriores; não consulta a grade adversária para escolher.

As três dificuldades são diferentes?

Ainda não. Médio e difícil compartilham o mesmo algoritmo.

Exercícios práticos

Explique o resultado de ler_coordenada(" b3 "): retorna (2, 1).

Desenhe as casas ocupadas por um navio vertical de tamanho 3 em (0, 2): (0, 2), (1, 2), (2, 2).

Simule uma tentativa de atirar duas vezes em A1 e identifique onde o ValueError é lançado.

Explique por que importar batalha_naval não chama menu(), mas executar o arquivo diretamente chama.

Percorra uma vitória e indique a ordem: atacar, mostrar resultado, verificar derrota, salvar resultado e retornar.

Ao estudar, tente justificar cada escolha com sua responsabilidade no jogo. Memorizar a sintaxe ajuda, mas compreender como os dados mudam em um turno é o que permite explicar o programa com segurança.